DRISHYAMITRA (Agentic Photos Evaluation and Segregation)

Project Description

DRISHYAMITRA (Agentic Photos Evaluation and Segregation) is an AI-powered photo management system designed to bring intelligence and automation to how users organize, search, and share their memories.

Unlike traditional photo galleries that rely on manual sorting, DRISHYAMITRA uses deep learning-based facial recognition, modern event-driven background queues, and natural language understanding to create an intuitive and efficient photo experience. Whether it's identifying people in images, organizing them into smart folders, or delivering them across platforms, the system ensures seamless photo handling with minimal human effort.

At its core, DRISHYAMITRA integrates advanced computer vision models such as InsightFace (buffalo_l) to achieve highly accurate face detection and recognition. Once photos are uploaded, the system automatically detects and extracts face coordinates, mapping them against user-defined personas. This automation leads to intelligent organization - grouping photos into person-specific collections, maintaining a clean relational hierarchy, and enabling quick retrieval through tags or voice-like queries. The combination of AI-driven recognition and structured folder management makes DRISHYAMITRA both powerful and user-friendly.

Beyond recognition, DRISHYAMITRA incorporates a conversational AI chatbot assistant that enables natural interaction with your photo collection. Users can simply ask, "Show me photos of Priya from last month," or "Send John's pictures to WhatsApp," and the system interprets and executes the task using Groq-powered natural language processing. The agent loop is built with multi-model routing (using llama-3.1-8b-instant for low-latency queries and llama-3.3-70b-versatile for complex multi-step tool calls) and rate-limit/parsing fail-safes. This multimodal interaction turns photo management into an intelligent dialogue, extending functionality beyond search into smart actions like automated sharing, batch delivery, and history tracking.

Built with a modern polyglot microservice architecture, DRISHYAMITRA uses Node.js with Express for its backend API orchestration, Python Flask for its computer vision Face Service, React.js (Vite) for its frontend, Redis/BullMQ for asynchronous job queues, and MongoDB Atlas for database storage. It features secure token-based authentication, encrypted database records, and direct integrations with email (Nodemailer SMTP) and WhatsApp (via whatsapp-web.js). DRISHYAMITRA is more than a gallery—it's a personalized photo companion that combines AI, automation, and communication to redefine how users manage and interact with their digital memories.

Scenarios & Use Cases

Scenario 1: Family Photo Organization & Memory Management

Riya, a working professional, has over 15,000 photos collected from years of vacations, family events, and celebrations. Her photo collection is spread across her phone, Google Drive, and an old laptop, making it nearly impossible to find specific pictures quickly.

After setting up DRISHYAMITRA, she uploads her entire photo library through the web interface. The system automatically scans every image, detects faces, and recognizes family members she's labeled before - like "Mom," "Dad," "Ananya," and "Grandma." New faces are flagged for labeling, allowing Riya to tag relatives or friends once, after which DRISHYAMITRA learns and remembers them for future uploads. The system then organizes all photos into neatly labeled collections, like "Family Trips," "Weddings," or "Festivals."

Later, when Riya wants to reminisce, she simply types or says, "Show me photos of Grandma from Diwali 2022," and within seconds, the AI retrieves exactly those photos. She can even ask, "Email Mom her birthday photos from last year," and the chatbot automatically attaches and sends them via Gmail. What once took hours now happens effortlessly - transforming Riya's digital chaos into an intelligent, organized memory archive.

Scenario 2: Event Photographer Workflow Automation

Aarav, a wedding and event photographer, deals with thousands of photos after every event. Sorting, tagging, and delivering photos to clients often takes days. By integrating DRISHYAMITRA into his post-production workflow, Aarav automates most of this process.

Once the event photos are uploaded, DRISHYAMITRA detects all faces and identifies recurring individuals (like the couple, families, and guests). It groups photos automatically - for example, "Bride & Groom," "Family Group," "Candid Moments," and more. Aarav can label key people once, and the system remembers them for all future shoots, improving with each upload.

Through the built-in chatbot, he can simply command, "Show me all photos of the couple during the ceremony," or "Send family group photos to client@example.com." Since event batches frequently exceed standard attachment constraints (e.g., 25MB for Gmail), DRISHYAMITRA automatically triggers a Socket.io confirmation modal asking Aarav if he wishes to compile and deliver the batch as a compressed archive. Once confirmed, DRISHYAMITRA streams the photo files in parallel directly from Cloudinary, compresses them in-memory, uploads the raw ZIP back to Cloudinary, sends the PR download proxy link via email or WhatsApp, and logs the history. A BullMQ worker cleans up the temporary ZIP file from Cloudinary 24 hours later, reclaiming storage automatically. This saves Aarav hours of effort and allows him to focus on editing and creativity instead of logistics.

Scenario 3: Corporate Media Management & Collaboration

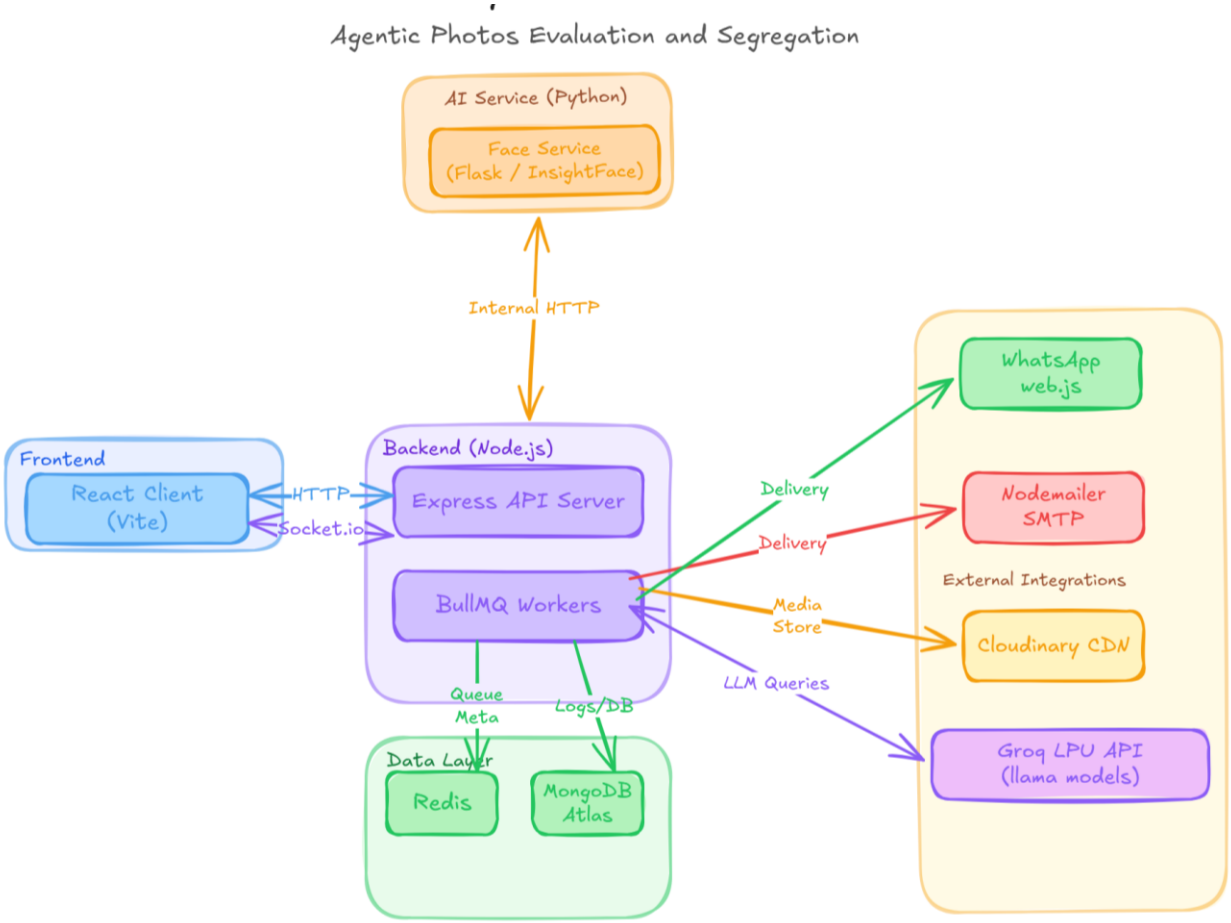
At a digital marketing agency, the creative team handles thousands of images for different campaigns and clients. These include event photos, product launches, and press coverage. Over time, managing and retrieving these visuals becomes a challenge, especially when multiple teams are involved. The company deploys DRISHYAMITRA as its central AI-powered media management platform.

Whenever team members upload campaign photos, the system automatically detects people, identifies employees, clients, and public figures, and organizes them into structured folders like "Client A - Launch 2024," "Team Events," or "Brand Photoshoots."

When the marketing lead, Neha, needs visuals for a quarterly report, she doesn't dig through shared drives. Instead, she asks the chatbot, "Find all photos of Rohan and Priya from the Client A launch event," and DRISHYAMITRA instantly retrieves them. She can then say, "Email them to our PR team," and the AI handles it seamlessly. The delivery is logged automatically in the MongoDB Delivery History collection for compliance and tracking. This automation not only saves time but also enhances collaboration, security, and brand consistency across projects - making DRISHYAMITRA an indispensable tool for modern media teams.

Architecture

DRISHYAMITRA is designed as a polyglot microservice system. The architecture features decoupled frontend and backend elements that operate asynchronously to process high-throughput image analytics workflows.



Pre-requisites

- **1. Python Environment Setup:** Install Python 3.9+ (required for insightface and scientific packages) and initialize a virtual environment inside the face-service/ directory.
- **2. Node.js Installation:** Install Node.js 18+ (includes npm 9+) to support Vite, Express, ES modules, and modern JavaScript dependencies.
- **3. Backend Dependencies:** Initialize and install package manager dependencies inside the backend/ directory by executing npm install.
- **4. Face Service Dependencies:** Install required scientific and framework packages inside the face-service/ directory using pip install -r requirements.txt.

- **5. Database Setup:** Configure a MongoDB database named 'Drishyamitra' (either locally or via MongoDB Atlas cloud cluster).
- **6. Backend Configuration:** Create a .env file in the backend/ directory configuring Port, Database connections, Redis server, Groq API credentials, Cloudinary API access, Gmail credentials, and tokens.
- **7. Face Service Configuration:** Create a .env file in the face-service/ directory specifying the development port (defaulting to 5001).
- **8. Frontend Configuration:** Create a .env file in the frontend/ directory specifying system variables such as VITE_API_URL=http://localhost:5000 and VITE_SOCKET_URL=http://localhost:5000.
- **9. Backend Launch:** Run npm run dev to start the Node.js Express server, and execute python app.py (after activating venv) to start the Python Flask face-service.
- **10. Frontend Launch:** Start the React client with Vite by running npm run dev in the frontend/ directory.
- **11. Verification:** Confirm backend and Socket.io endpoints are on http://localhost:5000, Flask Face Service is on http://localhost:5001, and Vite UI client is accessible on http://localhost:5173.

Project Workflow

Milestone 1: Environment Setup and Project Initialization

Activity 1.1: Create and activate a Python virtual environment using python -m venv venv.

Activity 1.2: Set up the project folder structure for backend (Express), face-service (Flask), and frontend (React.js/Vite).

Activity 1.3: Configure .env files for backend, face-service, and frontend with required API keys, database URLs, and Cloudinary keys.

Activity 1.4: Install dependencies using npm install and pip install.

Activity 1.5: Verify environment setup and ensure reproducible builds.

Milestone 2: Backend API Development using Express (Node.js)

Activity 2.1: Develop RESTful API endpoints for photo management, face labeling, and chatbot functionalities.

Activity 2.2: Define Mongoose database schemas for User, Photo, Face, Person, ChatHistory, and Delivery History.

Activity 2.3: Implement database initialization with Mongoose connection and ensure schema consistency.

Activity 2.4: Create modular controllers for core services - Photo Ingestion, Face Processing, Chat Assistant, and Delivery Integration.

Activity 2.5: Implement authentication using JWT-based token access and secure password

---

Milestone 3: AI and Service Integration (InsightFace, Groq, Gmail, WhatsApp, and BullMQ)

Activity 3.1: Integrate Python Face Service with InsightFace (buffalo_l) models for robust face detection and vector embeddings generation.

Activity 3.2: Develop the AI Chat Assistant using Groq API with fallback routing and manual tool parsing.

Activity 3.3: Implement Email Delivery with Nodemailer SMTP and smart ZIP compilation.

Activity 3.4: Integrate WhatsApp delivery using whatsapp-web.js and size-based checks.

Activity 3.5: Build background queues (BullMQ) for asynchronous processing, recognition, and asset cleanup.

Milestone 4: React.js Vite Frontend Development

Activity 4.1: Design a responsive and intuitive dashboard-style user interface with React, Vite, and Tailwind CSS v4.0.

Activity 4.2: Implement photo upload dropzone, face labeling overlay canvas, and photo gallery components with state management.

Activity 4.3: Develop the conversational chatbot interface with dynamic tool result grid cards.

Activity 4.4: Integrate Socket.io for real-time ingestion status and delivery progress notifications.

Activity 4.5: Configure environment variables with VITE_API_URL and optimize Vite builds for deployment.

Milestone 5: Testing and Deployment

Activity 5.1: Conduct backend unit testing for face services, agent loop, and email/WhatsApp functions.

Activity 5.2: Perform frontend testing for component rendering, route accessibility, and chat flow validation.

Activity 5.3: Validate backend-frontend integration and ensure accurate real-time data synchronization.

Activity 5.4: Perform end-to-end output validation, delivery tracking audits, and storage reclamation tests.

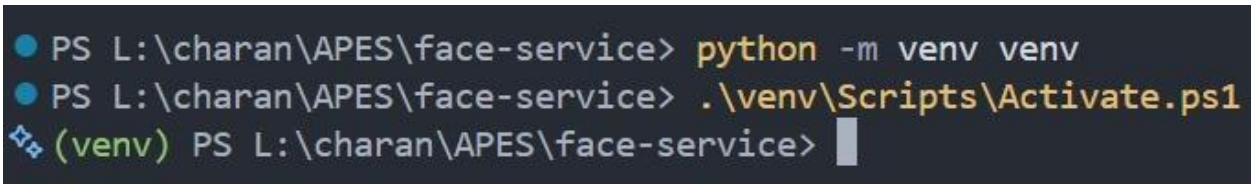
Milestone 1: Environment Setup and Project Initialization

Activity 1.1: Create and activate a Python virtual environment using python -m venv venv.

To isolate Python libraries for the computer vision components, navigate to the face-service/ directory. Create a new virtual environment using the command python -m venv venv. Once generated, activate it using the appropriate terminal command:

- **Windows PowerShell:** venv\Scripts\Activate.ps1
- **Windows Command Prompt:** venv\Scripts\activate.bat
- **macOS/Linux Shell:** source venv/bin/activate

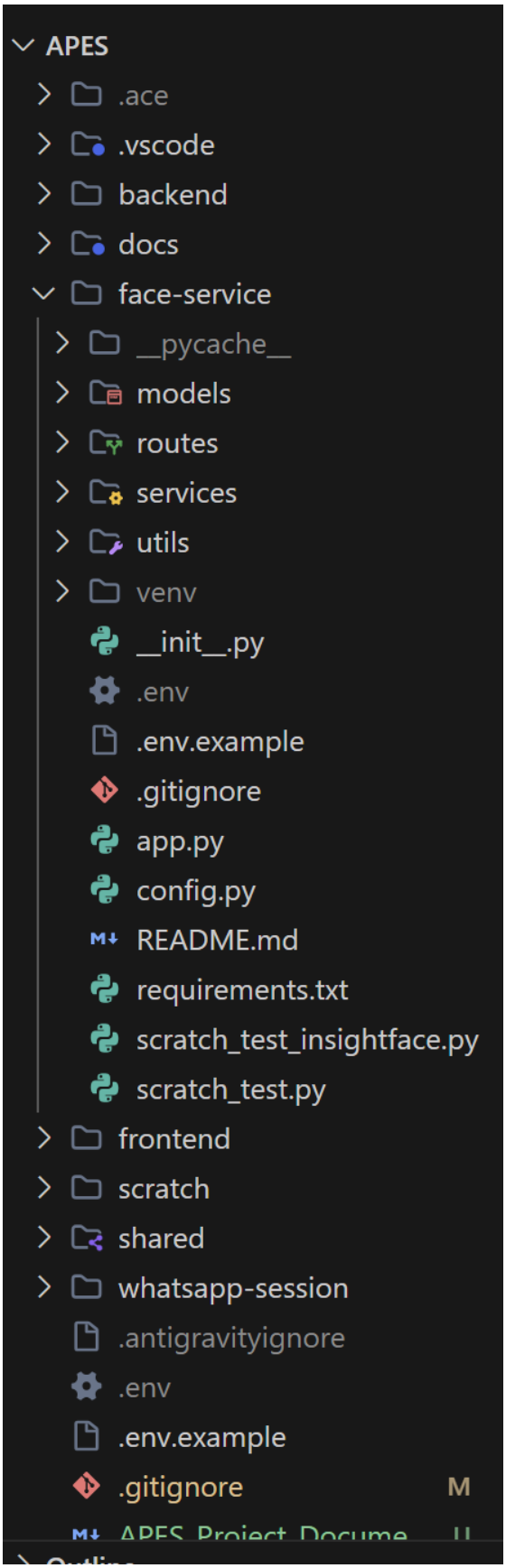
This isolates dependencies like insightface, onnxruntime, and OpenCV, avoiding global library version conflicts.



Activity 1.2: Set up the project folder structure for backend (Express), face-service (Flask), and frontend (React.js/Vite).

Organize the project into logical sub-repositories to establish separation of concerns:

- **backend/:** Node.js Express workspace containing controllers, database models, background BullMQ workers, routes, services, and configuration.
- **face-service/:** Python Flask service dedicated to running the ONNX models for face analysis.
- **frontend/:** React single page application initialized with Vite, using Tailwind CSS v4.0 for styling.
- **shared/:** Shared assets, utilities, and helper functions.
- **docs/:** System documentation, API contracts, design decisions, and system feature maps.


APES
.ace
.vscode
backend
docs
face-service
__pycache__
models
routes
services
utils
venv
__init__.py
.env
.env.example
.gitignore
app.py
config.py
README.md
requirements.txt
scratch_test_insightface.py
scratch_test.py
frontend
scratch
shared
whatsapp-session
.antigravityignore
.env
.env.example
.gitignore
M

Activity 1.3: Configure .env files for backend, face-service, and frontend with required API keys, database URLs, and Cloudinary keys.

To secure application keys and parameters, setup isolated environment files:

- **backend/.env:** Declare fields for port configurations (PORT), database URIs (MONGO_URI), Redis caches (REDIS_URL), Groq tokens (GROQ_API_KEY), Cloudinary URLS (CLOUDINARY_URL), Nodemailer accounts (GMAIL_EMAIL, GMAIL_PASSWORD), and JWT security secrets.
- **face-service/.env:** Configure ports (PORT=5001).
- **frontend/.env:** Configure Vite client endpoints (VITE_API_URL=http://localhost:5000 and VITE_SOCKET_URL=http://localhost:5000).

Ensure all .env files are appended to .gitignore to prevent leaking credentials. The core services integrated include Groq API, Gmail API / SMTP Nodemailer, WhatsApp Web API (whatsapp-web.js), and the InsightFace Model (buffalo_l).

```
# Port for the Express backend server
PORT=5000

# MongoDB Connection String (Atlas or Local)
MONGO_URI=mongodb+srv://<username>:<password>@cluster0.mongodb.net/drishyamitra?retryWrites=true&w=majority

# Redis connection string for BullMQ and Session memory
REDIS_URL=redis://localhost:6379

# Groq API key for llama-3 models
GROQ_API_KEY=gsk_your_groq_api_key_here

# Cloudinary credentials for photo hosting
CLOUDINARY_CLOUD_NAME=your_cloudinary_cloud_name
CLOUDINARY_API_KEY=your_cloudinary_api_key
CLOUDINARY_API_SECRET=your_cloudinary_api_secret

# Email service settings (Gmail SMTP with App Password)
GMAIL_USER=your_email@gmail.com
GMAIL_APP_PASS=your_gmail_app_password
    Ctrl+L for Agent
# Authentication settings
JWT_SECRET=your_jwt_secret_key_here
JWT_EXPIRES_IN=7d

# Environment identifier
NODE_ENV=development
CLIENT_URL=http://localhost:5173

# Path to persist whatsapp-web.js authenticated session
WHATSAPP_SESSION_PATH=./whatsapp-session

# Agent loop settings
MAX_TOOL_DEPTH=5
MAX_HISTORY=20

# Python face microservice URL
# Local development:
FACE_SERVICE_URL=http://localhost:5001
# Railway production — use internal service URL from Railway dashboard:
# FACE_SERVICE_URL=https://face-service.railway.internal

# Threshold for matching face embeddings (cosine similarity)
FACE_MATCH_THRESHOLD=0.72
```

Activity 1.4: Install dependencies using npm install and pip install.

Execute package installers in each workspace environment:

- **Backend:** Run npm install to install dependencies such as express, mongoose, redis, bullmq, socket.io, archiver, and nodemailer.

```
PS L:\charan\APES\backend> npm install
npm warn deprecated uuid@9.0.1: uuid@10 and below is no longer supported.  For ESM codebases, update to uuid@latest.  For CommonJS codebases, use
uid@11 (but be aware this version will likely be deprecated in 2028).
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been f
xed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated fluent-ffmpeg@2.1.3: Package no longer supported. Contact Support at https://www.npmjs.com/support for more info.

added 451 packages, and audited 452 packages in 1m

57 packages are looking for funding
  run `npm fund` for details

1 moderate severity vulnerability

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

- **Face Service:** Run pip install -r requirements.txt (inside the virtual environment) to install libraries like flask, insightface, onnxruntime, opencv-python, and numpy.

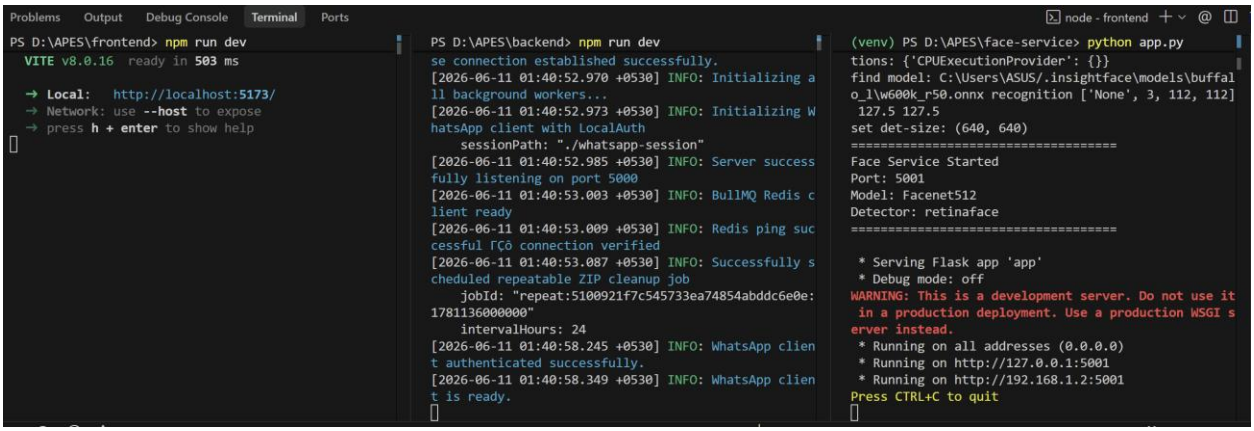
```
(venv) PS L:\charan\APES\face-service> pip install -r requirements.txt
Collecting Flask (from -r requirements.txt (line 1))
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting flask-cors (from -r requirements.txt (line 2))
  Downloading flask_cors-6.0.5-py3-none-any.whl.metadata (5.4 kB)
Collecting opencv-python (from -r requirements.txt (line 3))
  Downloading opencv_python-4.13.0.92-cp37-abi3-win_amd64.whl.metadata (20 kB)
Collecting gunicorn (from -r requirements.txt (line 4))
  Downloading gunicorn-26.0.0-py3-none-any.whl.metadata (5.4 kB)
Collecting numpy (from -r requirements.txt (line 5))
  Downloading numpy-2.4.6-cp312-cp312-win_amd64.whl.metadata (6.6 kB)
Collecting python-dotenv (from -r requirements.txt (line 6))
  Using cached python_dotenv-1.2.2-py3-none-any.whl.metadata (27 kB)
Collecting insightface (from -r requirements.txt (line 7))
  Downloading insightface-1.0.1-py3-none-any.whl.metadata (11 kB)
Collecting onnxruntime (from -r requirements.txt (line 8))
  Downloading onnxruntime-1.26.0-cp312-cp312-win_amd64.whl.metadata (5.5 kB)
Collecting requests (from -r requirements.txt (line 9))
```

- **Frontend:** Run npm install to load React, Tailwind CSS v4.0, Axios, Socket.io-client, Lucide React, and Framer Motion.


```
PS L:\charan\APES\backend> npm install
npm warn deprecated uuid@9.0.1: uuid@10 and below is no longer supported.  For ESM codebases, update to uuid@latest.  For CommonJS codebases, use
uid@11 (but be aware this version will likely be deprecated in 2028).
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been f
xed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated fluent-ffmpeg@2.1.3: Package no longer supported. Contact Support at https://www.npmjs.com/support for more info.

added 451 packages, and audited 452 packages in 1m

57 packages are looking for funding
  run `npm fund` for details

1 moderate severity vulnerability

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

Activity 1.5: Verify environment setup and ensure reproducible builds.

Verify the environment configurations by booting the service instances synchronously: Start the Flask Face Service via python app.py, start the Express API Server using npm run dev, and start the React Frontend via npm run dev. Confirm files like package-lock.json and requirements lists are tracked in git to lock exact dependency chains.



Milestone 2: Backend API Development using Express (Node.js)

Activity 2.1: Develop RESTful API endpoints for photo management, face labeling, and chatbot functionalities.

Create modular Express controllers mapped under the /api/v1 namespace. Core endpoints include:

- **POST /api/v1/photos/upload:** Handles uploaded multipart/form-data images, sends files to Cloudinary, inserts a photo model status as 'processing', and adds face-recognition jobs.
- **GET /api/v1/photos:** Retrieves all uploaded photos.
- **POST /api/v1/faces/label:** Associates an unlabeled face centroid with a named Person document, triggering cascading label updates.
- **POST /api/v1/chat:** Exposes chatbot reasoning interface, feeding history to the Groq agent loop.
- **GET /api/v1/delivery/download/:deliveryId:** Proxy-streams compiled ZIP files directly from Cloudinary raw assets to download endpoints.

```
import { Router } from "express";
import { uploadPhoto, getPhotos, deletePhoto, bulkDeletePhotos, getPhotoDetails, getI
import { authMiddleware } from "../middlewares/auth.middleware.js";
import { uploadSingle } from "../middlewares/upload.middleware.js";
import { validatePhotoId } from "../validators/photo.validator.js";
import { getIO } from "../socket/index.js";
import { emitRecognitionProgress, emitRecognitionDone } from "../socket/events.js";

const router = Router();

// Apply authMiddleware globally to all photo routes
router.use(authMiddleware);

router.post("/upload", uploadSingle, uploadPhoto);
router.get("/", getPhotos);
router.get("/stats", getPhotoStats);
router.get("/:id", validatePhotoId, getPhotoDetails);
router.delete("/:id", validatePhotoId, deletePhoto);
router.post("/bulk-delete", bulkDeletePhotos);

// Simulation route for Sprint 3.5 verification
router.post("/test-progress-trigger", (req, res) => {
  const io = getIO();
  const userId = req.user._id;
  const { photoId } = req.body;
  const jobId = "sim-job-" + Math.random().toString(36).substring(2, 9);

  let progress = 0;
  const interval = setInterval(() => {
    if (progress < 100) {
      emitRecognitionProgress(io, userId, { jobId, progress, photoId });
      progress += 25;
    } else {
      emitRecognitionProgress(io, userId, { jobId, progress: 100, photoId });
      clearInterval(interval);
      setTimeout(() => {
        emitRecognitionDone(io, userId, {
          success: true,
          jobId,
          photoId,
          totalFaces: 2,
          matchedFaces: 1,
          unknownFaces: 1
        });
      }, 1000);
    }
  }, 1000);

  res.json({ success: true, jobId });
});

export default router;
```

```
import { Router } from "express";
import { getUnlabeledFaces, labelFace, getFaceSuggestion, getLabeled
import { authMiddleware } from "../middlewares/auth.middleware.js";

const router = Router();

// Apply authMiddleware globally to all face management routes
router.use(authMiddleware);

router.get("/unlabeled", getUnlabeledFaces);
router.post("/:faceId/label", labelFace);
router.get("/:faceId/suggest", getFaceSuggestion);
router.get("/people", getLabeledPeople);
router.get("/people/:personId/photos", getPersonPhotos);

export default router;
```

```
import express from "express";
import { handleChat, clearChat } from "../controllers/chat.controller.js";
import { authMiddleware } from "../middlewares/auth.middleware.js";
import { asyncHandler } from "../utils/asyncHandler.js";

const router = express.Router();

router.post("/", authMiddleware, asyncHandler(handleChat));
router.delete("/", authMiddleware, asyncHandler(clearChat));

export default router;
```

```
import express from "express";
import { getDeliveryHistoryHandler, downloadZipHandler } from "../controllers/delivery.controller.js";
import { authMiddleware } from "../middlewares/auth.middleware.js";
import { asyncHandler } from "../utils/asyncHandler.js";

const router = express.Router();

router.get("/history", authMiddleware, asyncHandler(getDeliveryHistoryHandler));
router.get("/download/:deliveryId", asyncHandler(downloadZipHandler));


export default router;
```

Activity 2.2: Define Mongoose database schemas for User, Photo, Face, Person, ChatHistory, and Delivery History.

Implement relational mappings inside the database layer using Mongoose models:

- **User:** Holds accounts, email addresses, password hashes, and profile timestamps.
- **Photo:** Stores Cloudinary URLs, file identifiers, original byte sizes, related face object reference arrays, user reference, and processing states (processing, completed, error).
- **Face:** Stores bounding box coordinates (x, y, w, h), 512-dimension floating-point face vector embeddings, parent photo ID, associated named Person ID, and identification flags (isLabeled).
- **Person:** Holds defined names (e.g. "Mom"), creator user references, and centroid embeddings (average vector profile).
- **ChatHistory:** Logs chat exchanges between users and the agent.
- **Delivery History:** Tracks photo shares, email/WhatsApp recipients, zip statuses (pending, queued, delivered), Cloudinary ZIP public IDs, URLs, and cleanup timestamps.

```
import mongoose from "mongoose";

const ChatHistorySchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: [true, "User association is required"]
  },
  sessionId: {
    type: String,
    required: [true, "Session ID is required"]
  },
  summary: {
    type: String,
    default: ""
  },
  userMessage: {
    type: String,
    required: [true, "User message is required"]
  },
  assistantReply: {
    type: String,
    required: [true, "Assistant reply is required"]
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

// Index to retrieve chat history chronologically for a user
ChatHistorySchema.index({ userId: 1, createdAt: -1 });

const ChatHistory = mongoose.model("ChatHistory", ChatHistorySchema);
export default ChatHistory;
```

```
import mongoose from "mongoose";

const DeliveryHistorySchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: [true, "User association is required"]
  },
  recipient: {
    type: String,
    required: [true, "Recipient details are required"],
    trim: true
  },
  medium: {
    type: String,
    enum: ["email", "whatsapp"],
    required: [true, "Delivery medium is required"]
  },
  photoIds: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: "Photo"
  }],
  count: {
    type: Number
  },
  format: {
    type: String,
    enum: ["links", "zip"]
  },
  zipUrl: {
    type: String
  },
  cloudinaryPublicId: {
    type: String
  },
  messageId: {
    type: String
  },
  deliveredAt: {
    type: Date
  },
  zipDeletedAt: {
    type: Date
  },
  status: {
    type: String,
    enum: ["queued", "delivered", "failed"],
    default: "queued"
  },
  error: {
    type: String,
    default: null
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

// Index to retrieve user's delivery audit log chronologically
DeliveryHistorySchema.index({ userId: 1, createdAt: -1 });

const DeliveryHistory = mongoose.model("DeliveryHistory", DeliveryHistorySchema);
export default DeliveryHistory;
```

```
import mongoose from "mongoose";

const FaceSchema = new mongoose.Schema({
  photoId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Photo",
    required: [true, "Photo association is required"]
  },
  personId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Person",
    default: null
  },
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: [true, "User association is required"]
  },
  embedding: {
    type: [Number],
    required: [true, "Embedding is required"],
    validate: {
      validator: function(val) {
        return val && val.length === this.embeddingDimension;
      },
      message: function() {
        return `Embedding length must exactly match the embeddingDimension (${this ?
      }
    }
  },
  embeddingDimension: {
    type: Number,
    required: true,
    default: 512
  },
  bbox: {
    x: { type: Number, required: [true, "Bounding box x coordinate is required"] },
    y: { type: Number, required: [true, "Bounding box y coordinate is required"] },
    w: { type: Number, required: [true, "Bounding box width is required"] },
    h: { type: Number, required: [true, "Bounding box height is required"] }
  },
  isLabeled: {
    type: Boolean,
    default: false
  },
  labelSource: {
    type: String,
    enum: ["manual", "propagation"],
    default: null
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

// Indexes
FaceSchema.index({ personId: 1 });
FaceSchema.index({ photoId: 1 });
FaceSchema.index({ isLabeled: 1 });
FaceSchema.index({ userId: 1 });

const Face = mongoose.model("Face", FaceSchema);
export default Face;
```

```
import mongoose from "mongoose";

const PersonSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: [true, "User association is required"]
  },
  name: {
    type: String,
    required: [true, "Person name is required"],
    trim: true
  },
  nameNormalized: {
    type: String,
    required: [true, "Normalized name is required"],
    lowercase: true,
    trim: true
  },
  centroid: {
    type: [Number],
    default: null
  },
  centroidCount: {
    type: Number,
    default: 0
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

// Compound unique index to prevent duplicate named persons for the same
PersonSchema.index({ userId: 1, nameNormalized: 1 }, { unique: true });

const Person = mongoose.model("Person", PersonSchema);
export default Person;
```

```
import mongoose from "mongoose";

const emailRegex = /^\S+@\S+\.\S+$/;

const UserSchema = new mongoose.Schema({
  username: {
    type: String,
    required: [true, "Username is required"],
    trim: true
  },
  email: {
    type: String,
    required: [true, "Email is required"],
    unique: true,
    trim: true,
    lowercase: true,
    match: [emailRegex, "Please provide a valid email address"]
  },
  passwordHash: {
    type: String,
    required: [true, "Password hash is required"]
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

const User = mongoose.model("User", UserSchema);
export default User;
```

Activity 2.3: Implement database initialization with Mongoose connection and ensure schema consistency.

Create database connection configuration utilities. Connect to MongoDB inside the boot file index.js using mongoose.connect(process.env.MONGO_URI). Define schemas uniformly. Any index adjustments or constraint modifications are deployed synchronously at boot.

```
import mongoose from "mongoose";
import { env } from "./env.js";
import { logger } from "./logger.js";

export const connectDB = async () => {
  try {
    mongoose.connection.on("connected", () => {
      logger.info("MongoDB database connection established successfully.");
    });

    mongoose.connection.on("error", (err) => {
      logger.error(`MongoDB connection error: ${err.message}`);
    });

    mongoose.connection.on("disconnected", () => {
      logger.warn("MongoDB database connection lost/disconnected.");
    });

    await mongoose.connect(env.MONGO_URI);
  } catch (error) {
    logger.error(`Failed to connect to MongoDB on startup: ${error.message}`);
    process.exit(1);
  }
};
```

.

Activity 2.4: Create modular controllers for core services - Photo Ingestion, Face Processing, Chat Assistant, and Delivery Integration.

Encapsulate the business logic within controllers inside backend/src/controllers/:

- **photo.controller.js:** Coordinates file parses, writes to Cloudinary CDN, creates document instances, and triggers BullMQ recognition jobs.
- **face.controller.js:** Serves face coordinates, lists unlabeled crops, manages manual annotations, and runs label propagation.
- **chat.controller.js:** Wraps session loads, agent loop executions, and JSON returns.

- **delivery.controller.js:** Processes download proxies and audit requests.

```
export const uploadPhoto = asyncHandler(async (req, res) => {
  if (!req.file) {
    return errorResponse(res, 400, "No file uploaded");
  }

  logger.info({ requestId: req.id, userId: req.user._id }, "Uploading buffer stream to Cloudinary");

  // Call Cloudinary stream upload
  const uploadResult = await photoService.uploadStream(req.file.buffer);

  // Persist image metadata
  const photo = new Photo({
    userId: req.user._id,
    url: uploadResult.secure_url,
    cloudinaryPublicId: uploadResult.public_id,
    width: uploadResult.width,
    height: uploadResult.height,
    bytes: uploadResult.bytes,
    status: "completed",
    originalName: req.file.originalname
  });

  await photo.save();

  // Clear stats cache
  redis.del(`stats:${req.user._id}`).catch(err => logger.error({ err: err.message }, "Failed to clear stats cache"));

  logger.info({ requestId: req.id, photoId: photo._id }, "Photo registered in MongoDB");

  // Trigger face recognition & persistence synchronously
  try {
    logger.info({ requestId: req.id, photoId: photo._id }, "Triggering face recognition for uploaded photo");
    const recognitionResult = await recognizeFaces(photo.url);
    if (recognitionResult && recognitionResult.faces) {
      const summary = await processRecognizedFaces(photo._id, recognitionResult.faces);
      photo.faceCount = summary.processed;
      await photo.save();
      logger.info(
        { requestId: req.id, photoId: photo._id, faceCount: photo.faceCount },
        "Face recognition completed and faceCount updated"
      );
    }
  } catch (error) {
    logger.error(
      { requestId: req.id, photoId: photo._id, err: error.message },
      "Face recognition processing failed during upload"
    );
  }
```

```
if (typeof personName !== "string" || personName.trim().length === 0) {
  logger.warn(
    {
      endpoint: "POST /faces/:faceId/label",
      userId,
      faceId,
      duration: Date.now() - startTime
    },
    "Label face attempt failed: invalid or empty personName input"
  );
  return errorResponse(res, 400, "Person name is required and must be a non-empty string");
}

try {
  const result = await faceLabelingService.labelFace(faceId, userId, personName);

  // Clear stats cache
  redis.del(`stats:${userId}`).catch(err => logger.error({ err: err.message }, "Failed to clear

  const duration = Date.now() - startTime;

  logger.info(
    {
      endpoint: "POST /faces/:faceId/label",
      userId,
      faceId,
      duration
    },
    "Face successfully labeled and propagated"
  );

  return res.status(200).json(result);
} catch (err) {
  const duration = Date.now() - startTime;

  logger.error(
    {
      endpoint: "POST /faces/:faceId/label",
      userId,
      faceId,
      duration,
      error: err.message
    },
    "Error occurred during face labeling controller execution"
  );

  // Map custom exception objects to their respective HTTP status codes
  if (err instanceof ValidationError) {
    return errorResponse(res, 400, err.message);
  }
  if (err instanceof AuthorizationError) {
    return errorResponse(res, 403, err.message);
  }
  if (err instanceof NotFoundError) {
    return errorResponse(res, 404, err.message);
  }

  // Default status for unexpected server errors
  return errorResponse(res, 500, "Unexpected server error");
```

```
export async function handleChat(req, res) {
  const rawMessage = req.body.message;

  // 1. Validation: Reject undefined, null, or non-string inputs
  if (rawMessage === undefined || rawMessage === null) {
    return errorResponse(res, 400, "Message is required.");
  }

  if (typeof rawMessage !== "string") {
    return errorResponse(res, 400, "Message must be a string.");
  }

  // Trim whitespace
  const message = rawMessage.trim();

  // Reject empty or whitespace-only messages
  if (message === "") {
    return errorResponse(res, 400, "Message cannot be empty.");
  }

  // 2. Extract authenticated user ID string from JWT user payload
  const userId = req.user._id.toString();

  // 3. Call runAgent
  try {
    const result = await runAgent({
      userId,
      message
    });

    // 4. Format response to presentation-ready data
    const formattedResponse = formatAgentResponse(result);

    // 5. Return reply and cards wrapped in standardized successRe
    return successResponse(
      res,
      formattedResponse,
      "Chat response generated successfully."
    );
  } catch (error) {
    logger.error(
      { err: error, userId, message },
      "Chat controller failed to process request"
    );

    return errorResponse(
      res,
      500,
      "Failed to process chat request."
    );
  }
}
```

```
/**
 * Handles GET /api/v1/delivery/download/:deliveryId
 * Streams the extensionless ZIP file from Cloudinary and returns it with a correct .zip extension.
 */
export async function downloadZipHandler(req, res) {
  const { deliveryId } = req.params;

  if (!mongoose.Types.ObjectId.isValid(deliveryId)) {
    return res.status(400).send("Invalid delivery ID format.");
  }

  try {
    const deliveryRecord = await DeliveryHistory.findById(deliveryId);
    if (!deliveryRecord) {
      return res.status(404).send("Delivery history record not found.");
    }

    if (deliveryRecord.format !== "zip" || !deliveryRecord.zipUrl) {
      return res.status(400).send("This delivery is not in ZIP format or the ZIP URL is missing.");
    }

    if (deliveryRecord.zipDeletedAt) {
      return res.status(410).send("This ZIP archive has expired and is no longer available.");
    }

    logger.info({ deliveryId, zipUrl: deliveryRecord.zipUrl }, "Proxying ZIP download from Cloudinary");

    // Fetch the raw stream from Cloudinary
    const response = await axios({
      method: "get",
      url: deliveryRecord.zipUrl,
      responseType: "stream"
    });

    // Set response headers to force download with proper filename
    res.setHeader("Content-Type", "application/zip");
    res.setHeader("Content-Disposition", `attachment; filename="apes_photos_${deliveryId}.zip"`);

    // Stream error handling
    response.data.on("error", (err) => {
      logger.error({ err: err.message, deliveryId }, "Error streaming ZIP file from Cloudinary");
      if (!res.headersSent) {
        res.status(500).send("Error downloading file.");
      }
    });

    // Pipe the stream to the response
    response.data.pipe(res);
  } catch (error) {
    logger.error({ error: error.message, deliveryId }, "Failed to download ZIP file");
    return res.status(500).send("Failed to download ZIP file.");
  }
}
```

Activity 2.5: Implement authentication using JWT-based token access and secure password hashing.

Ensure workspace routes are protected. When users submit registration or login commands, encrypt user passwords using bcrypt (with 10 hashing rounds) before saving them to the database. Authenticate users by signing JSON Web Tokens (JWT) containing user identifiers and security payloads. Protect API routers using authentication middleware that validates token payloads from Request headers before executing controllers.

```
/**
 * Handle user credential authentication
 */
export const login = asyncHandler(async (req, res) => {
  const { email, password } = req.body;
  const lowercaseEmail = email.toLowerCase().trim();

  const user = await User.findOne({ email: lowercaseEmail });
  if (!user) {
    return errorResponse(res, 401, "Invalid email or password");
  }

  const isMatch = await bcrypt.compare(password, user.passwordHash);
  if (!isMatch) {
    return errorResponse(res, 401, "Invalid email or password");
  }

  const token = generateToken(user._id, user.username);

  return successResponse(
    res,
    {
      token,
      expiresIn: env.JWT_EXPIRES_IN,
      user: {
        id: user._id,
        username: user.username,
        email: user.email
      }
    },
    "Login successful"
  );
});
```

Milestone 3: AI and Service Integration (InsightFace, Groq, Gmail, WhatsApp, and BullMQ)

Activity 3.1: Integrate Python Face Service with InsightFace (buffalo_l) models for robust face detection and vector embeddings generation.

Inside face-service/, configure the Flask endpoint /recognize which maps calls to face_service.py to perform the following: Receives photo URLs, downloads the file binary from Cloudinary, and reads it using OpenCV; Executes the InsightFace FaceAnalysis (name="buffalo_l") model to locate face objects and compute 512-dimension vector representations; Applies IoU-based Non-Maximum Suppression (NMS) to eliminate duplicate detection bounding boxes; Returns coordinates and embeddings to the Node.js background queue worker.

Inside the BullMQ worker: Calculates cosine similarity of the newly extracted embeddings against the user's existing Person centroids; Auto-labels faces with high similarity matches, and marks unknown profiles as unlabeled crops.

```python
def compute_iou(box1, box2):
    """
    Computes Intersection over Union (IoU) of two bounding boxes.
    Each box is in [x_min, y_min, x_max, y_max] format.
    """
    x_min1, y_min1, x_max1, y_max1 = box1
    x_min2, y_min2, x_max2, y_max2 = box2

    inter_x_min = max(x_min1, x_min2)
    inter_y_min = max(y_min1, y_min2)
    inter_x_max = min(x_max1, x_max2)
    inter_y_max = min(y_max1, y_max2)

    if inter_x_max <= inter_x_min or inter_y_max <= inter_y_min:
        return 0.0

    inter_area = (inter_x_max - inter_x_min) * (inter_y_max - inter_y_min)
    area1 = (x_max1 - x_min1) * (y_max1 - y_min1)
    area2 = (x_max2 - x_min2) * (y_max2 - y_min2)

    union_area = area1 + area2 - inter_area
    if union_area == 0:
        return 0.0

    return inter_area / union_area

def deduplicate_faces(faces, threshold=0.70):
    """
    Applies Non-Maximum Suppression (NMS) based on IoU threshold to deduplicate detections.
    Detections are sorted by det_score in descending order.
    """
    sorted_faces = sorted(faces, key=lambda f: getattr(f, "det_score", 0.0), reverse=True)

    keep = []
    for face in sorted_faces:
        discard = False
        for kept_face in keep:
            iou = compute_iou(face.bbox, kept_face.bbox)
            if iou > threshold:
                discard = True
                break
        if not discard:
            keep.append(face)
    return keep

def extract_embeddings(img):
    """
    Executes InsightFace represent function to extract face embeddings and bounding boxes.

    Args:
        img (numpy.ndarray): OpenCV BGR image array.

    Returns:
        list: List of detected face representation dictionaries.
    """
    try:
        if img is None or not hasattr(img, "shape"):
            raise ValueError("Input image is invalid or empty")

        faces = app.get(img)
        faces = deduplicate_faces(faces, threshold=0.70)

        # Get image dimensions for boundary clipping
        img_h, img_w = img.shape[:2]

        results = []
        for face in faces:
            # Check for None embedding or invalid bbox
            if face.embedding is None:
                continue
            if face.bbox is None or len(face.bbox) < 4:
                continue

            # bbox is [x_min, y_min, x_max, y_max]
            bbox = face.bbox
            x_min = max(0, int(bbox[0]))
```

Activity 3.2: Develop the AI Chat Assistant using Groq API with fallback routing and manual tool parsing.

Implement the agent orchestrator inside agentLoop.js:

- **Model Routing:** Evaluation logic routes conversational inputs to llama-3.1-8b-instant and complex actions to llama-3.3-70b-versatile.
- **Fallback Systems:** Catches 429 rate limit exceptions, automatically switching active contexts to alternative models.
- **Manual Regex Parser:** Incorporates a parser parseFailedGeneration to capture raw JSON tool call strings from gateway error outputs when standard JSON tokenization fails, ensuring seamless operation.

```
while (true) {
  // Audit and log the exact request payload sent to Groq
  logger.info({
    groqPayload: {
      model: activeModel,
      messages: groqMessages,
      tools: activeTools,
      tool_choice: "auto"
    }
  }, "Sending request payload to Groq");

  try {
    response = await groq.chat.completions.create({
      model: activeModel,
      messages: groqMessages,
      tools: activeTools,
      tool_choice: "auto",
      temperature: DEFAULT_TEMPERATURE
    });
    model = activeModel; // Update successfully used model
    break; // Success!
  } catch (err) {
    const isRateLimit = err.status === 429 ||
                        err.type === "RateLimitError" ||
                        err.name === "RateLimitError" ||
                        err.message?.includes("429") ||
                        err.message?.includes("rate_limit");

    if (isRateLimit) {
      rateLimitedModels.add(activeModel);
      logger.warn({ model: activeModel, error: err.message }, "Groq model hit rate limit.");

      // Determine fallback model
      const fallback = activeModel === MODELS.REASONING ? MODELS.FAST : MODELS.REASONING;
      if (rateLimitedModels.has(fallback)) {
        logger.error("All available models are rate-limited. Failing.");
        throw err;
      }

      logger.info({ fallback }, "Retrying with fallback model...");
      activeModel = fallback;
      continue; // Try again with fallback
    }

    const isToolUseError = err.status === 400 ||
                           err.message.includes("tool_use_failed") ||
                           err.message.includes("Failed to call a function");

    if (isToolUseError) {
      if (activeModel === MODELS.FAST && !rateLimitedModels.has(MODELS.REASONING)) {
        logger.warn({ err: err.message }, "Fast model tool execution failed due to Groq parser bug. Retrying with Reasoning model...");
        try {
          response = await groq.chat.completions.create({
            model: MODELS.REASONING,
            messages: groqMessages,
            tools: activeTools,
            tool_choice: "auto",
            temperature: DEFAULT_TEMPERATURE
          });
          model = MODELS.REASONING;
          break; // Success!
        } catch (retryErr) {
          const isRetryRateLimit = retryErr.status === 429 ||
                                   retryErr.type === "RateLimitError" ||
                                   retryErr.name === "RateLimitError" ||
                                   retryErr.message?.includes("429") ||
                                   retryErr.message?.includes("rate_limit");
          if (isRetryRateLimit) {
            rateLimitedModels.add(MODELS.REASONING);
          }

          const failedGen = retryErr.error?.failed_generation || retryErr.failed_generation || retryErr.message;
          const parsedCall = parseFailedGeneration(failedGen);
          if (parsedCall) {
            logger.info({ parsedCall }, "Reasoning model failed tool use. Recovered successfully via manual parsing of failed_generation");
            response = {
              choices: [
                {
                  message: {
                    role: "assistant",
                    content: null,
                    tool_calls: [
                      {
                        id: "call_" + Math.random().toString(36).substring(2),
                        type: "function",
                        function: {
                          name: parsedCall.name,
                          arguments: JSON.stringify(parsedCall.args)
```

Activity 3.3: Implement Email Delivery with Nodemailer SMTP and smart ZIP compilation.

Set up standard mailing utilities using Nodemailer SMTP. When sharing photo sets: Calculates cumulative payload file sizes from the database. If size stays below 25MB, dispatches direct files. If it exceeds 25MB, prompts user via Socket.io (delivery:zip-confirm) to compress the files. On confirmation, downloads photos, compiles a ZIP archive in-memory using archiver, uploads it to Cloudinary as a raw resource, and emails the proxy download endpoint link.

```
/**
 * Generates a ZIP archive in memory from Cloudinary-hosted photos and uploads it back to Cloudinary as a raw file.
 *
 * @param {object} params
 * @param {Array<object>} params.photos - Array of populated Photo documents containing at least `url` and `_id`.
 * @param {number} [params.concurrencyLimit=3] - Maximum number of concurrent downloads.
 * @returns {Promise<object>} Result containing zipUrl, cloudinaryPublicId, fileSize, and photoCount.
 */
export async function createZip({ photos, concurrencyLimit = 3 }) {
  // 1. Validation
  if (!photos || !Array.isArray(photos)) {
    throw new ZipServiceError("photos must be a valid array", { code: "INVALID_INPUT" });
  }
  if (photos.length === 0) {
    throw new ZipServiceError("photos array cannot be empty", { code: "INVALID_INPUT" });
  }

  // 2. Prepare task structures and determine filenames (with deduplication)
  const seenNames = new Set();
  const tasks = photos.map((photo, index) => {
    if (!photo || !photo.url || !photo._id) {
      throw new ZipServiceError("Invalid photo object: missing url or _id", {
        code: "INVALID_INPUT",
        photo
      });
    }

    let filename = "";
    if (photo.url) {
      try {
        const parsedUrl = new URL(photo.url);
        const pathname = parsedUrl.pathname;
        const basename = pathname.substring(pathname.lastIndexOf("/") + 1);
        if (basename && basename.includes(".") && !basename.startsWith(".")) {
          filename = basename;
        }
      } catch (err) {
        // Fallback if URL parsing fails
      }
    }

    // Fallback to photo-N.ext if filename could not be extracted
    if (!filename) {
      let ext = ".jpg";
      if (photo.url) {
        try {
          const parsedUrl = new URL(photo.url);
          const pathname = parsedUrl.pathname;
          const lastDot = pathname.lastIndexOf(".");
          if (lastDot !== -1) {
            const possibleExt = pathname.substring(lastDot).toLowerCase();
```

Activity 3.4: Integrate WhatsApp delivery using whatsapp-web.js and size-based checks.

Implement WhatsApp sharing via the whatsapp.service.js utility. The backend initializes whatsapp-web.js, writing authentication files locally and outputting QR codes to console setups on startup. When dispatches are requested, checks sizes against a 100MB threshold. Exceeding batches are compiled into raw ZIP files on Cloudinary and delivered as download proxy links.

```
  // Start initialization process
  client.initialize().catch((err) => {
    logger.error({ err: err.message }, "Error during WhatsApp client initialization");
  });

  return client;
}

export async function getWhatsAppStatus() {
  let clientState = null;
  if (client) {
    try {
      clientState = await client.getState();
    } catch (err) {
      clientState = `Error: ${err.message}`;
    }
  }
  return {
    initialized: !!client,
    isReadyLocal: isReady,
    clientState
  };
}

/**
 * Helper to check if the WhatsApp client is ready and connected.
 *
 * @returns {Promise<boolean>}
 */
export async function isWhatsAppReady() {
  if (!client) return false;
  try {
    const state = await client.getState();
    return state === "CONNECTED" || isReady;
  } catch (err) {
    // If getState fails (e.g. browser not open), return the local ready flag
    return isReady;
  }
}

/**
 * Sends a WhatsApp message containing photo links or custom text.
 *
 * @param {object} params
 * @param {string} params.recipient - The recipient's phone number.
 * @param {object[]} [params.photos] - Array of photo objects containing url/imageUrl.
 * @param {string} [params.text] - Custom text message body (takes priority if provided).
 * @returns {Promise<object>} Delivery metadata.
 */
export async function sendWhatsApp({ recipient, photos, text, zipUrl }) {
  if (!client) {
    throw new WhatsAppServiceError("WhatsApp client has not been initialized. Call initializ
```

Activity 3.5: Build background queues (BullMQ) for asynchronous processing, recognition, and asset cleanup.

Establish three separate queues inside Redis:

- **recognitionQueue:** Handles face ingestion tasks for newly uploaded photos.

```
/**
 * Add a face recognition processing job to the queue.
 * @param {object} data - Job payload: { photoId }
 * @param {string|object} data.photoId - MongoDB ObjectId of the photo to process
 * @returns {Promise<object>} The added or existing BullMQ Job instance
 */
export async function addRecognitionJob(data) {
  if (!data || !data.photoId) {
    throw new Error("Invalid job data. Required: photoId");
  }

  const photoIdStr = data.photoId.toString();

  try {
    // Enforce deduplication at the queue level by using the photoId as the BullMQ jobId
    const job = await recognitionQueue.add(
      "recognize-face",
      { photoId: photoIdStr },
      { jobId: photoIdStr }
    );
    logger.info({ jobId: job.id, photoId: photoIdStr }, "Successfully queued photo for recognition");
    return job;
  } catch (err) {
    logger.error({ err, photoId: photoIdStr }, "Failed to add job to recognition queue");
    throw err;
  }
}
```

- **deliveryQueue:** Offloads email and WhatsApp photo deliveries.

```
/**
 * Add a delivery task job to the queue.
 * @param {object} data - Job payload: { requestId }
 * @param {string} data.requestId - Unique identifier of the delivery request
 * @returns {Promise<object>} The added or existing BullMQ Job instance
 */
export async function addDeliveryJob(data, opts = {}) {
  if (!data || !data.requestId) {
    throw new Error("Invalid job data. Required: requestId");
  }

  const requestIdStr = data.requestId.toString();

  try {
    // Enforce deduplication at the queue level by using the requestId as the BullMQ jobId
    const job = await deliveryQueue.add(
      "deliver-photos",
      { requestId: requestIdStr },
      { jobId: requestIdStr, ...opts }
    );
    logger.info({ jobId: job.id, requestId: requestIdStr }, "Successfully queued delivery job");
    return job;
  } catch (err) {
    logger.error({ err, requestId: requestIdStr }, "Failed to add job to delivery queue");
    throw err;
  }
}
```

- **zipCleanupQueue:** Runs a cron job every 24 hours to identify expired raw ZIP assets in Cloudinary, call the Cloudinary destruction API, and clear database records to reclaim storage.

```
/**
 * Schedule the repeatable ZIP cleanup job to run at configured intervals.
 *
 * @returns {Promise<object>} The repeatable job info.
 */
export async function scheduleZipCleanup() {
  const intervalHours = env.ZIP_CLEANUP_INTERVAL_HOURS || 24;
  const intervalMs = intervalHours * 60 * 60 * 1000;

  try {
    // Add repeatable job to the queue
    const job = await cleanupZipQueue.add(
      "cleanup-expired-zips",
      {},
      {
        repeat: {
          every: intervalMs
        },
        jobId: "zip-cleanup-repeatable"
      }
    );
    logger.info({ jobId: job.id, intervalHours }, "Successfully scheduled repeatable ZIP cleanup job");
    return job;
  } catch (err) {
    logger.error({ err }, "Failed to schedule repeatable ZIP cleanup job");
    throw err;
  }
```

Milestone 4: React.js Vite Frontend Development

Activity 4.1: Design a responsive and intuitive dashboard-style user interface with React, Vite, and Tailwind CSS v4.0.

Build a modern interface layout in React initialized using Vite for rapid builds and asset processing. Styled using Tailwind CSS v4.0 for dark-mode features and animations. Set up standard layouts containing a navigation Sidebar, header actions, and alert toasts. Configure routes inside AppRouter.jsx to navigate between Dashboard, Gallery, Upload, Faces, and Chat.

```jsx
const AppRouter = () => {
  return (
    <BrowserRouter>
      <Routes>
        {/* Root redirect route */}
        <Route path="/" element={<RootRedirect />} />

        {/* Public only routes */}
        <Route element={<PublicRoute />}>
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
        </Route>

        {/* Protected routes */}
        <Route element={<ProtectedRoute />}>
          <Route element={<AppLayout />}>
            <Route path="/dashboard" element={<Dashboard />} />
            <Route path="/gallery" element={<Gallery />} />
            <Route path="/gallery/person/:personId" element={<PersonPhotosPage />} />
            <Route path="/upload" element={<Upload />} />
            <Route path="/chat" element={<Chat />} />
            <Route path="/faces" element={<FaceLabelingPage />} />
          </Route>
        </Route>

        {/* Fallback redirect */}
        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </BrowserRouter>
  );
};
export default AppRouter;
```

Activity 4.2: Implement photo upload dropzone, face labeling overlay canvas, and photo gallery components with state management.

Implement key interactive UI views:

- **Upload Dropzone:** Accepts files, displays drag animations, lists queue progress, and communicates ingestion status.
- **Gallery Grid:** Renders chronological card lists, supports photo selections, and includes detail modal overlays.
- **Face Labeling Canvas:** Draws face bounding boxes over images. Clicking boxes displays centroid suggestions and allows users to submit labels.

```
// Spotlight math calculating percentage overlays relative to natural dimensions
const getSpotlightStyles = () => {
  if (!naturalDimensions || !bbox) return { display: 'none' };

  const left = (bbox.x / naturalDimensions.width) * 100;
  const top = (bbox.y / naturalDimensions.height) * 100;
  const width = (bbox.w / naturalDimensions.width) * 100;
  const height = (bbox.h / naturalDimensions.height) * 100;

  return {
    left: `${left}%`,
    top: `${top}%`,
    width: `${width}%`,
    height: `${height}%`,
    position: 'absolute',
    border: '3px solid #c8501a',
    boxShadow: '0 0 0 9999px rgba(15, 14, 12, 0.7)', // Spotlight shader mask
    pointerEvents: 'none',
    borderRadius: '4px',
    zIndex: 10,
  };
};
```

```
{!loading && !error && naturalDimensions && (
  <div style={getContainerStyle()} className="shadow-2xl">
    <img
      src={photoUrl}
      alt="Full Context view"
      className="w-full h-full object-contain block"
    />
    {/* Highlight Overlay Spotlight */}
    <div style={getSpotlightStyles()} />
  </div>
)}
```

Activity 4.3: Develop the conversational chatbot interface with dynamic tool result grid cards.

Build the conversational client in Chat.jsx. It contains messaging input fields, typing notifications, and bubble layouts. Integrates custom result renderers. When the agent uses searchPhotos, the API includes metadata details in the response, and the chat renders interactive cards displaying photo matches directly in the conversation.

```
// Gallery message renderer
if (type === 'gallery') {
  return (
    <div className="flex flex-col w-full items-start my-3 select-text">
      <div className="w-full max-w-[85%] md:max-w-[90%]">
        <ToolResultGrid photos={content} />
      </div>
      {timeStr && (
        <span className="text-[10px] text-[#6b6760] font-mono mt-1 ml-2 select-none">
          {timeStr}
        </span>
      )}
    </div>
  );
}

// Standard Text message renderer
return (
  <div className={`flex flex-col w-full ${isUser ? 'items-end' : 'items-start'} my-2`}
    <div
      className={`max-w-[75%] px-4 py-3 rounded-2xl text-sm leading-relaxed whitespace
        isUser
          ? 'bg-[#c8501a] text-white rounded-br-none'
          : 'bg-[#f2f0eb] text-[#0f0e0c] rounded-bl-none border border-[#e8e4dc]'
      }`}
    >
      {content}
    </div>
    {timeStr && (
      <span
        className={`text-[10px] text-[#6b6760] font-mono mt-1 select-none ${
          isUser ? 'mr-2' : 'ml-2'
        }`}
      >
        {timeStr}
      </span>
    )}
  </div>
);
```

Activity 4.4: Integrate Socket.io for real-time ingestion status and delivery progress notifications.

Initialize Socket.io listeners inside the React views. Subscribes to backend events:

- **recognition:progress:** Updates progress bars on the upload dropzone.
- **delivery:zip-confirm:** Displays confirmation prompts if attachment sizes exceed limits.
- **delivery:started / delivery:done:** Displays success notifications to users.

```
  // Register event listeners
  socket.on(SOCKET_EVENTS.RECOGNITION_PROGRESS, handleProgress);
  socket.on(SOCKET_EVENTS.FACE_NEW, handleFaceNew);
  socket.on(SOCKET_EVENTS.RECOGNITION_DONE, handleDone);
  socket.on(SOCKET_EVENTS.DELIVERY_STARTED, handleDeliveryStarted);
  socket.on(SOCKET_EVENTS.DELIVERY_DONE, handleDeliveryDone);
  socket.on(SOCKET_EVENTS.DELIVERY_FAILED, handleDeliveryFailed);

  // Clean up event listeners on unmount or when socket instance changes
  return () => {
    socket.off(SOCKET_EVENTS.RECOGNITION_PROGRESS, handleProgress);
    socket.off(SOCKET_EVENTS.FACE_NEW, handleFaceNew);
    socket.off(SOCKET_EVENTS.RECOGNITION_DONE, handleDone);
    socket.off(SOCKET_EVENTS.DELIVERY_STARTED, handleDeliveryStarted);
    socket.off(SOCKET_EVENTS.DELIVERY_DONE, handleDeliveryDone);
    socket.off(SOCKET_EVENTS.DELIVERY_FAILED, handleDeliveryFailed);
  };
}, [socket]);
```

Activity 4.5: Configure environment variables with VITE_API_URL and optimize Vite builds for deployment.

Create a local env file declaring VITE_API_URL. Configure client builds: Use import.meta.env.VITE_API_URL to point Axios calls dynamically to backend ports. Execute npm run build to compile minimized index packages. Use code-splitting, tree-shaking, and asset compression to optimize payload delivery.

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
  ],
})
```

Milestone 5: Testing and Deployment

Activity 5.1: Conduct backend unit testing for face services, agent loop, and email/WhatsApp functions.

Implement automated test cases for the Express backend and Python face service: Create unit tests simulating InsightFace runs on mock images to verify bounding boxes; Simulate agent loop requests, mocking Groq API outputs to verify model routing, rate-limit fallbacks, and token calculations; Use Sinon or Jest mocks to simulate SMTP transfers and WhatsApp client sends without executing real network calls.

```
PS D:\APES\backend> node scripts/verifyModelRouter.js
[2026-06-11 05:19:38.140 +0530] INFO: Starting model router verification tests...
[2026-06-11 05:19:38.140 +0530] INFO: Testing input: "Show my photos"
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel("Show my photos") to be "llama-3.1-8b-instant", got "llama-3.1-8b-instant"
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: "Show photos of Dad and Mom"
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel("Show photos of Dad and Mom") to be "llama-3.1-8b-instant", got "llama-3.1-8b-instant"
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: "Show photos after 2023"
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel("Show photos after 2023") to be "llama-3.1-8b-instant", got "llama-3.1-8b-instant"
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: ""
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel("") to be "llama-3.1-8b-instant", got "llama-3.1-8b-instant"
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: "     "
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel("     ") to be "llama-3.1-8b-instant", got "llama-3.1-8b-instant"
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: null
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel(null) to be "llama-3.1-8b-instant", got "llama-3.1-8b-instant"
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: undefined
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel(undefined) to be "llama-3.1-8b-instant", got "llama-3.1-8b-instant"
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: 12345
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: "Show my photos and email them"
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel("Show my photos and email them") to be "llama-3.3-70b-versatile", got "llama-3.3-70b-versatile"
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: "Find Dad's birthday pictures and send them to Mom on WhatsApp"
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel("Find Dad's birthday pictures and send them to Mom on WhatsApp") to be "llama-3.3-70b-versatile", got "llama-3.3-70b-versatile"
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: "compress all files"
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel("compress all files") to be "llama-3.3-70b-versatile", got "llama-3.3-70b-versatile"
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: "first find photos then mail them"
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel("first find photos then mail them") to be "llama-3.3-70b-versatile", got "llama-3.3-70b-versatile"
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: "archive this session"
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel("archive this session") to be "llama-3.3-70b-versatile", got "llama-3.3-70b-versatile"
[2026-06-11 05:19:38.141 +0530] INFO: Testing input: "zip the photos"
[2026-06-11 05:19:38.141 +0530] INFO: [Assertion Passed] Expected selectModel("zip the photos") to be "llama-3.3-70b-versatile", got "llama-3.3-70b-versatile"
[2026-06-11 05:19:38.141 +0530] INFO: ALL MODEL ROUTER VERIFICATION TESTS COMPLETED SUCCESSFULLY!
```

Activity 5.2: Perform frontend testing for component rendering, route accessibility, and chat flow validation.

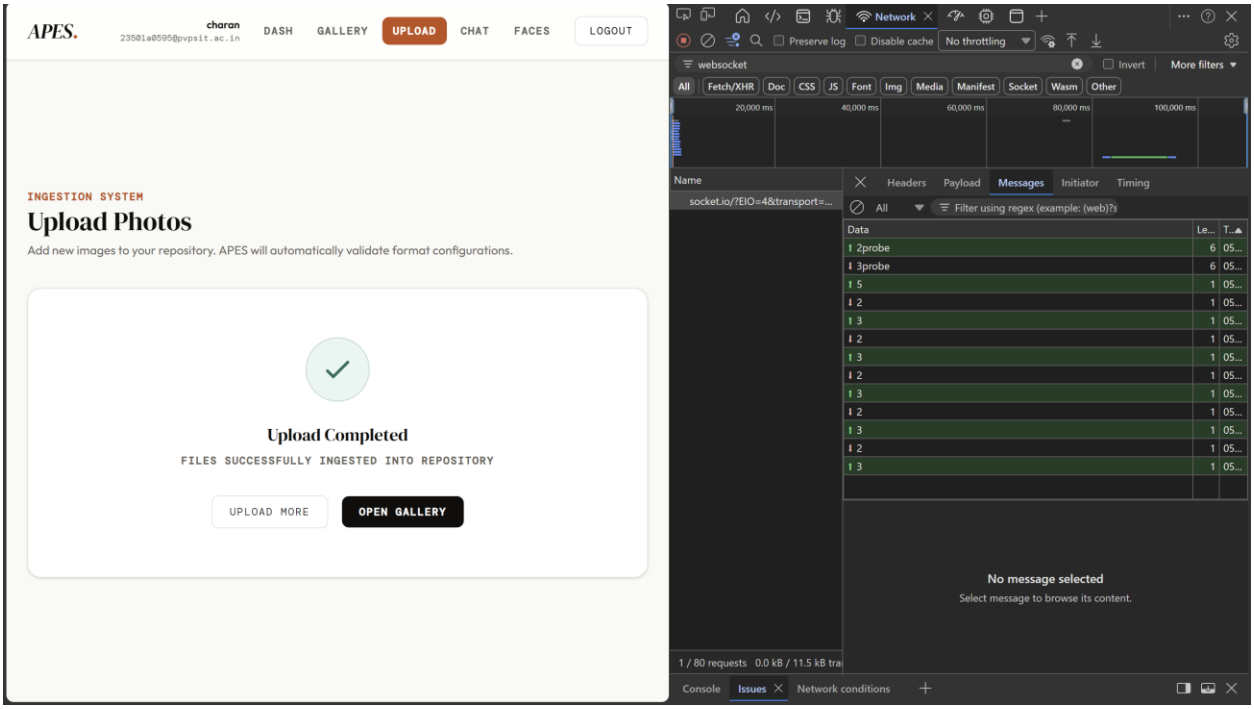
Verify user views using Jest or React Testing Library: Confirm Login and Dashboard views load. Test that clicking gallery uploads triggers post requests. Validate that Socket.io state messages render notifications correctly.

```
// 2. Call the hook to simulate mounting
const mockHandlers = {
  onFaceNew: () => console.log("Mock face new")
};

useSocketEvents(mockSocket, mockHandlers);

// Verify that useEffect blocks were registered
assert(effects.length === 2, `Expected 2 useEffect registrations, got ${effects.length}

// Execute effect 1 (handlers ref sync)
effects[0].effect();
assert(refStore.ref.current === mockHandlers, "Handlers ref failed to sync");

// Execute effect 2 (socket listeners registration)
const cleanupFn = effects[1].effect();

const expectedEvents = [
  "recognition:progress",
  "face:new",
  "recognition:done",
  "delivery:done",
  "delivery:failed"
];

// Check all registered events
for (const ev of expectedEvents) {
  assert(registered[ev] === 1, `Expected event "${ev}" to be registered exactly once, g
}

console.log("Success: All 5 socket event listeners registered successfully on mount.");
```

```
PS D:\APES\frontend> node scratch/test_hook.js
Starting React hook useSocketEvents unit tests...
Success: All 5 socket event listeners registered successfully on mount.
Success: Re-rendering did not register duplicate event listeners.
Success: All 5 socket event listeners removed successfully on unmount.
All hook unit tests passed successfully!
```

Activity 5.3: Validate backend-frontend integration and ensure accurate real-time data synchronization.

Perform integration checks across all modules: Upload a test photo from the React upload page; Confirm the progress bar rises in response to recognition:progress socket updates; Open database collections and verify coordinates match the image faces; Verify the new profiles appear in the face labeling gallery view.



Activity 5.4: Perform end-to-end output validation, delivery tracking audits, and storage reclamation tests.

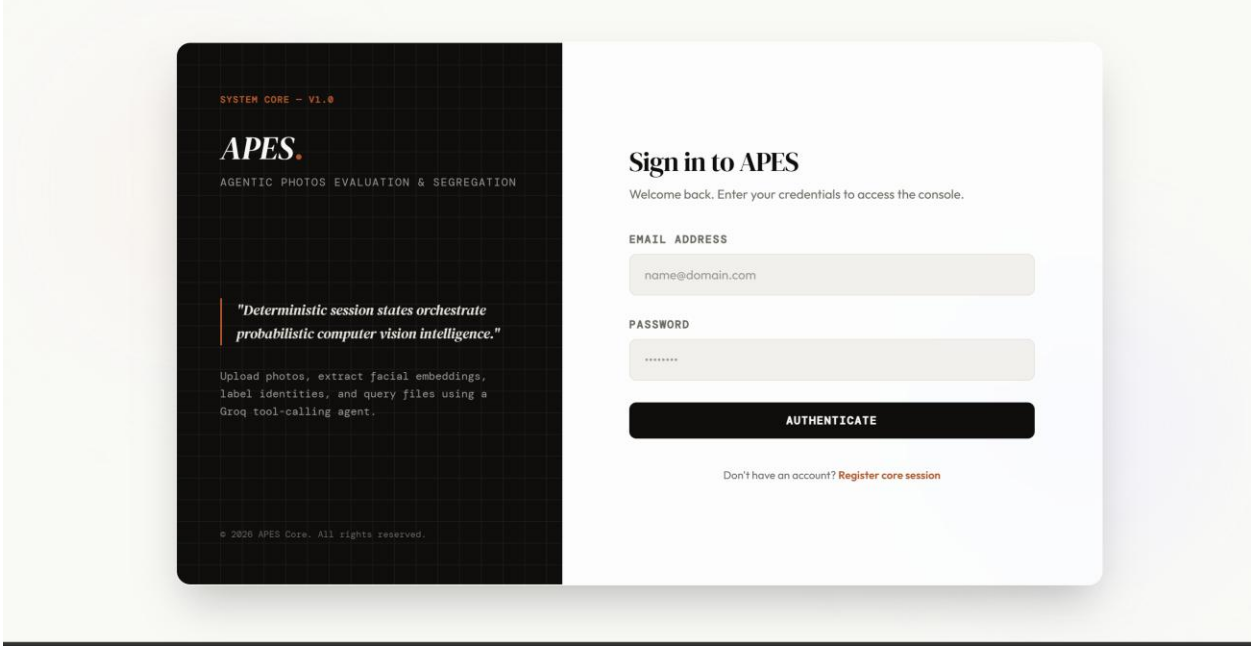
Verify sharing flows and cron operations: Trigger oversized delivery requests via the chatbot, confirm zip creations via prompt modals, and check email inboxes for zip proxy links. Access MongoDB databases using Compass to check that Delivery History documents log transactions correctly. Manually run BullMQ zip cleanup jobs to confirm expired zip files are successfully deleted from Cloudinary.

```
PS D:\APES\backend> node scratch/check_deliveries.js
Latest deliveries:
[
  {
    "_id": "6a29a07b92c33c633fd8070a",
    "userId": "6a268c108425277e3ddee488",
    "recipient": "+91 63012 65964",
    "medium": "whatsapp",
    "photoIds": [
      "6a29171a41120fd01e8fbdce",
      "6a2911405068584cb257a8a8",
      "6a2910f35068584cb257a743",
      "6a2910b15068584cb257a56a",
      "6a290f3b5068584cb2579948",
      "6a290efc5068584cb25798fd",
      "6a290e9d5068584cb25798fa",
      "6a290e825068584cb25798e5",
      "6a290e715068584cb25798d0",
      "6a290e5a5068584cb25798b7",
      "6a290dfc5068584cb25798b4",
      "6a290d9e5068584cb25798b1",
      "6a290cdf514a07faa47ca321",
      "6a290cadfeeba4d4b5ead50a",
      "6a290c4dfeeba4d4b5ead507",
      "6a290bd2feeba4d4b5ead1a9",
      "6a290bcbfeeba4d4b5ead1a0",
      "6a290bc4feeba4d4b5ead197",
      "6a290bbafeeba4d4b5ead18c",
      "6a290bb3feeba4d4b5ead183"
    ],
    "format": "zip",
    "zipUrl": "https://res.cloudinary.com/dxgl7wq2e/raw/upload/fl_attachment:shared_photos%252E
zip/v1781112955/apes/deliveries/delivery-0fe3dcc0-fb78-4252-9c2c-45c4a5c87bb8",
    "cloudinaryPublicId": "apes/deliveries/delivery-0fe3dcc0-fb78-4252-9c2c-45c4a5c87bb8",
    "status": "delivered",
    "error": "WhatsApp client has not been initialized. Call initializeWhatsApp() first.",
    "createdAt": "2026-06-10T17:35:55.517Z",
    "__v": 0,
```

User Interface Page Views

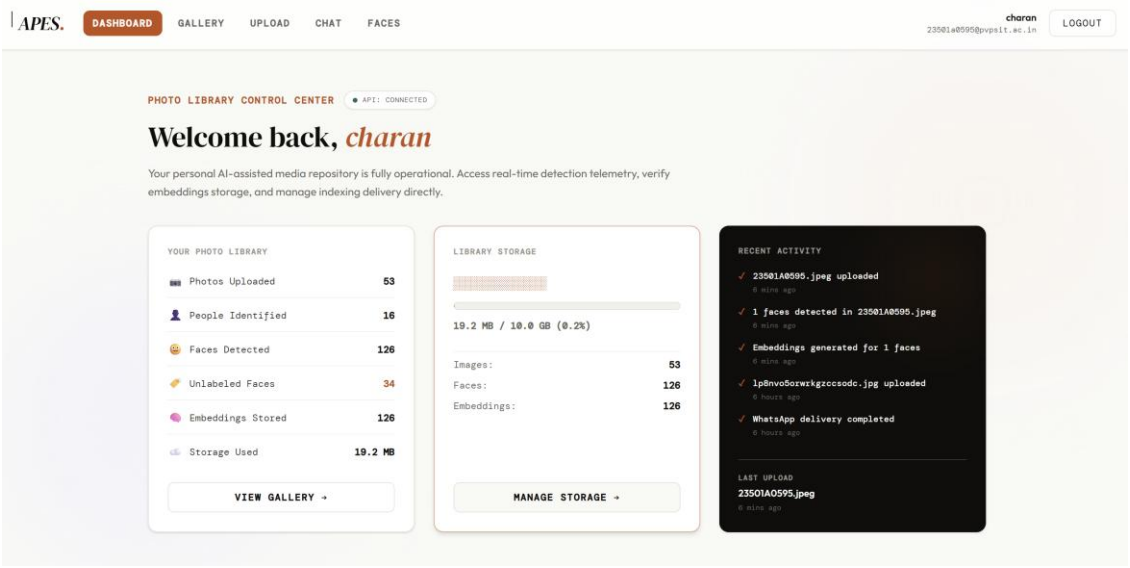
Home Page (Login and SignUp page)

The Home Page acts as the entry point to the DRISHYAMITRA system. It presents responsive Login and SignUp forms styled with Tailwind CSS, utilizing JWT-based session security.



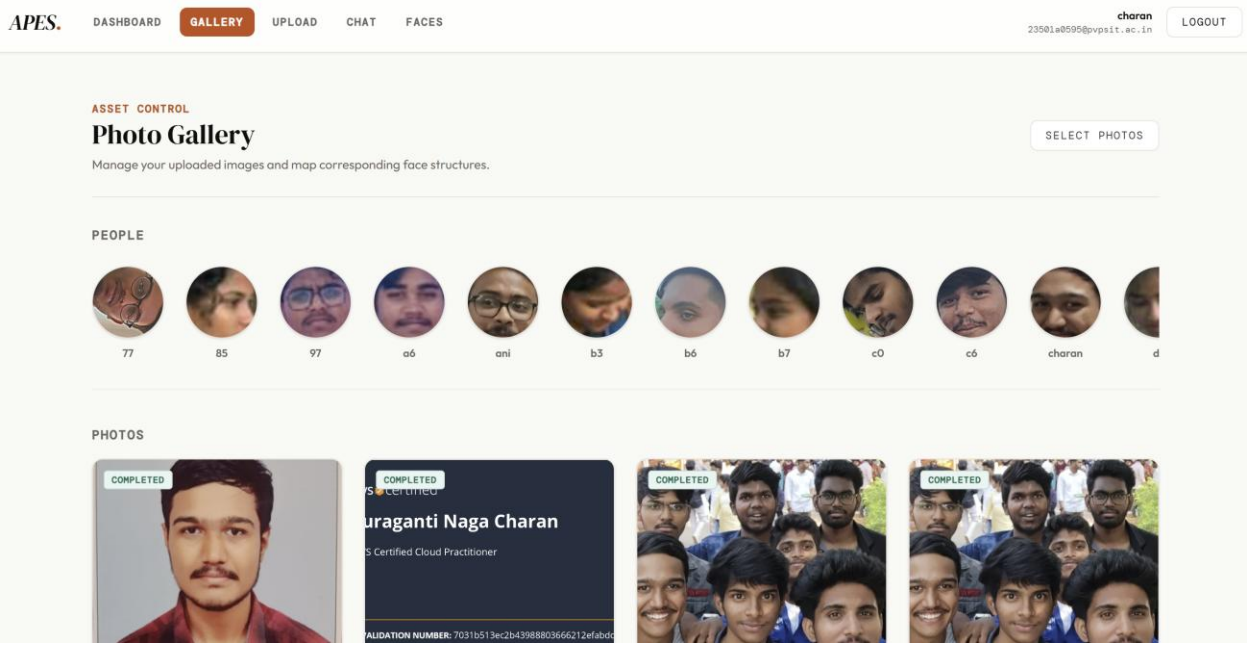
Dashboard Page

The DRISHYAMITRA Dashboard provides a centralized command center for users, showing current socket status, library statistics, and visual storage widgets.



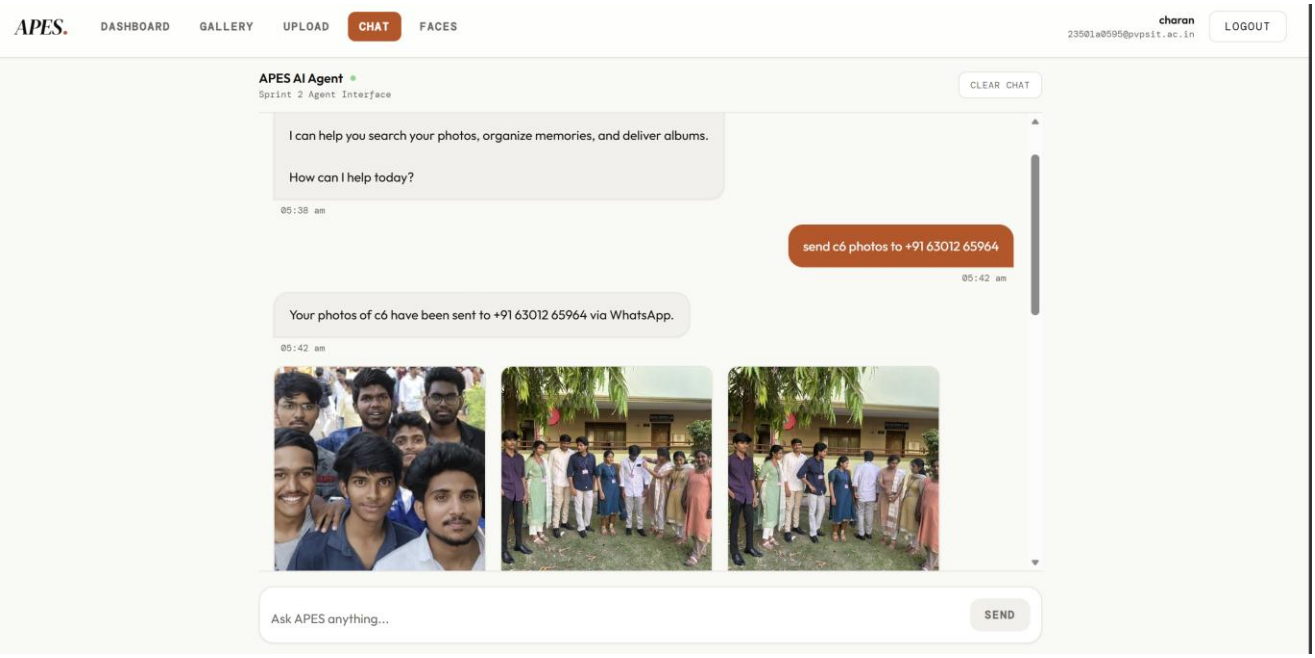
Gallery Page

The Gallery Page displays all uploaded photos in a chronological grid, featuring selection tools, deletion modals, and carousel overlays.



Chatbot Page

The Chatbot interface allows users to search, organize, and share their media collections using natural language chat commands.



Conclusion

The successful implementation of the DRISHYAMITRA (Agentic Photos Evaluation and Segregation) system demonstrates the power of combining modern containerized polyglot microservices with machine learning and agentic workflows. By integrating the high-performance InsightFace (buffalo_l) face detection models with a Node.js Express orchestrator and React Vite client, the system achieves fast and accurate face detection, embedding generation, and automatic categorization.

Key architectural innovations - such as Redis-backed background queues using BullMQ, automated in-memory ZIP streaming, and robust multi-model Groq API routing with fallback structures - ensure that the application remains responsive, secure, and resilient under heavy workloads. By addressing the manual organization bottleneck of photo collections, DRISHYAMITRA bridges the gap between advanced deep learning and everyday utility, delivering an intelligent photo companion that recognizes, organizes, and shares memories with precision and ease.